Parallelism in ML

Agenda

- Transformer architecture (the LLM dependency graph)
- Executing this dependency graph in parallel, efficiently

Large Language Models

You might have heard of them

How do they work?

How do we make them run quickly?

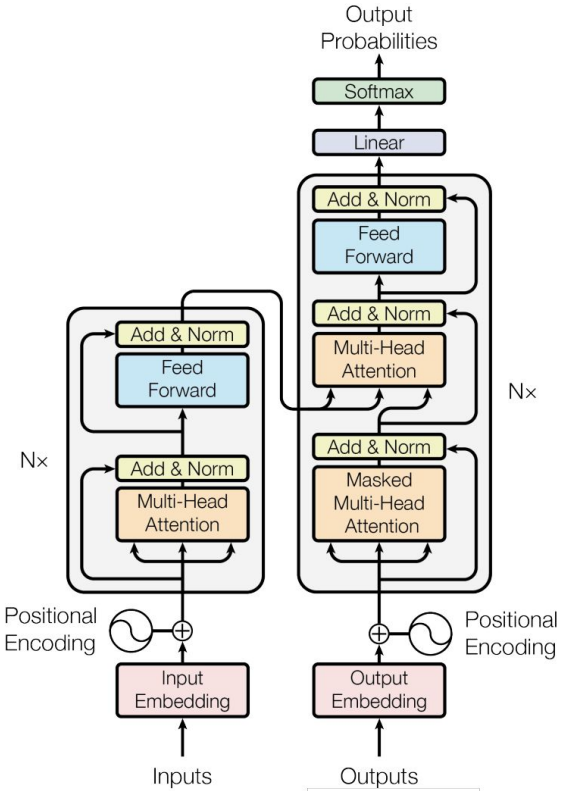
Transformer Architecture

ML is full of jargon!

This is just a dependency graph.

BERT

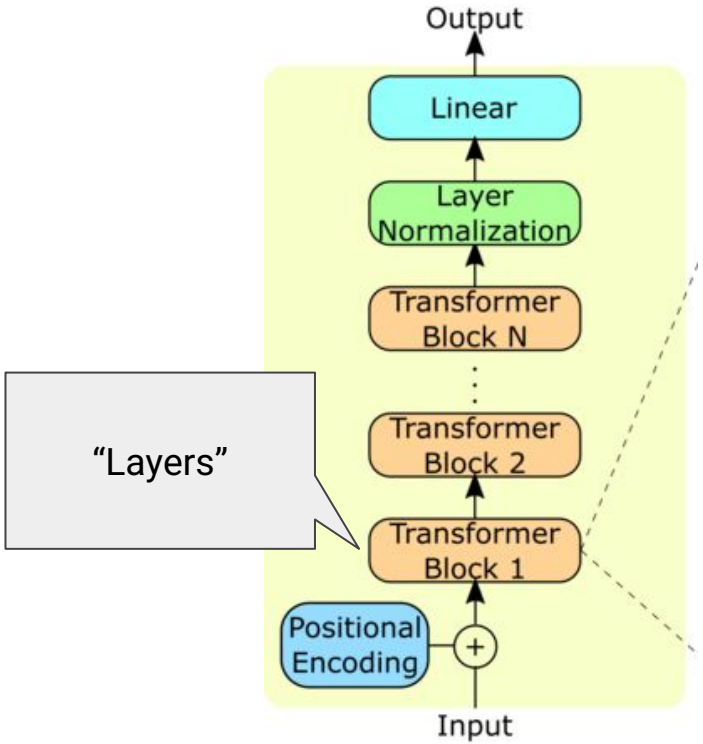
Encoder



GPT

Decoder

GPT 2 Model


Output
Linear
Layer Normalization
Transformer Block N
Transformer Block 2
Transformer Block 1
Positional Encoding
+
Input
“Layers”

The Basic Idea

- GPT is all just matrix multiplication
- Turn input words into vectors of floats
- Multiply them by a lot of matrices

Matrices contain “weights” or “parameters” — preset values

- Training is the act of determining these matrices
- Inference is just running some input through to get an output

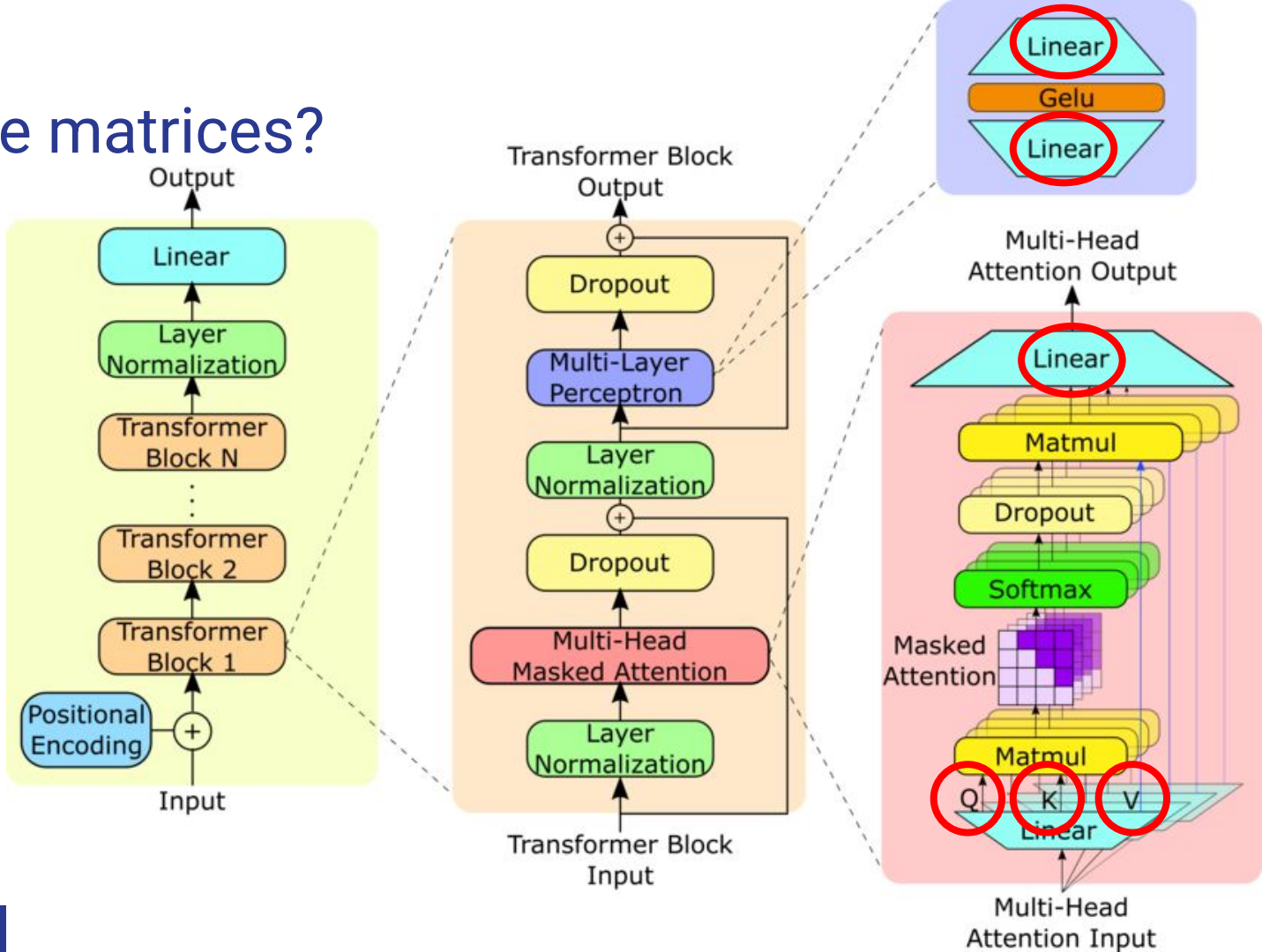
How many matrices?

Each “layer” involves ~6 matrices:

- Q, K, V, O, up, down

Typically ~32 layers, so ~200 matrices total that we need to multiply our input by

Where are the matrices?



How many matrices?

Each “layer” involves ~6 matrices:

- Q, K, V, O, up, down

Typically ~32 layers, so ~200 matrices total that we need to multiply our input by

Q, K, V, O are square, from llama3’s 4096x4096 to GPT3’s 12,228x12,228 elements

- 12,288 x 12,288 -> 151M floats -> 604MB per matrix

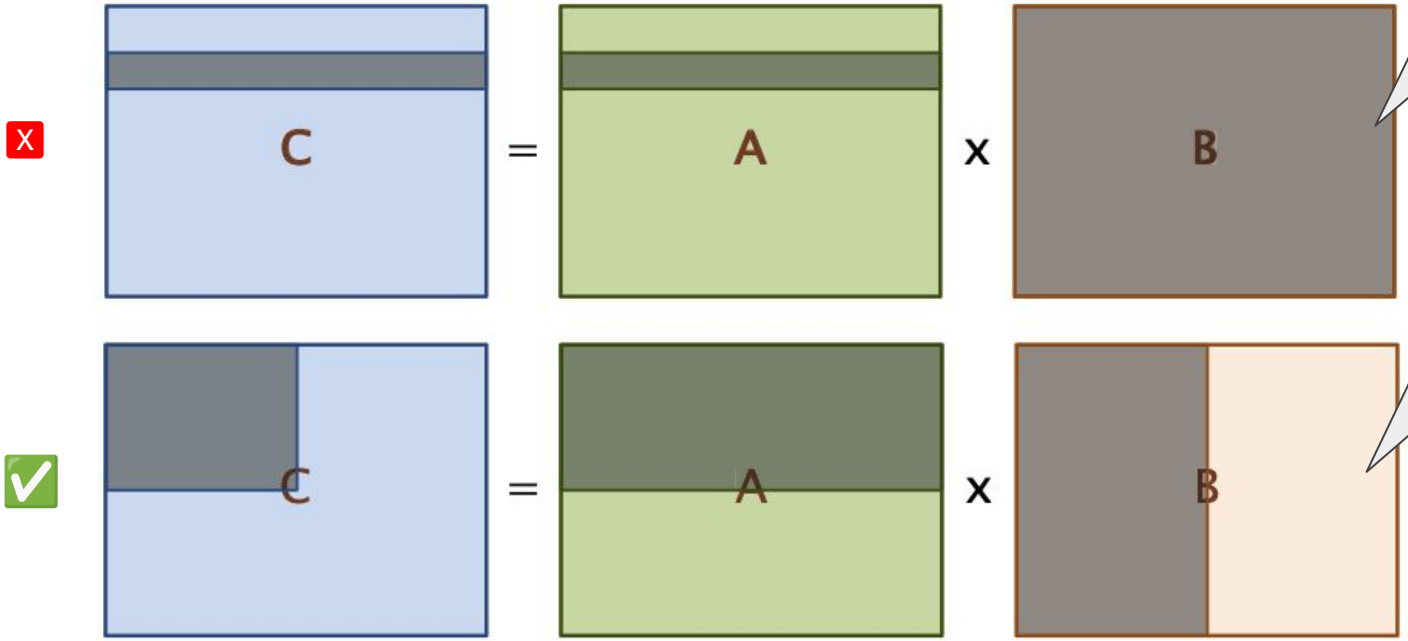
up, down are 4 times bigger -> 604M floats -> 2.4GB per matrix

Parallelism step 0: Run on a GPU

Matrix multiplication is embarrassingly parallel

Parallelism step 0: Run on a GPU

Matrix multiplication is highly optimizable (351)



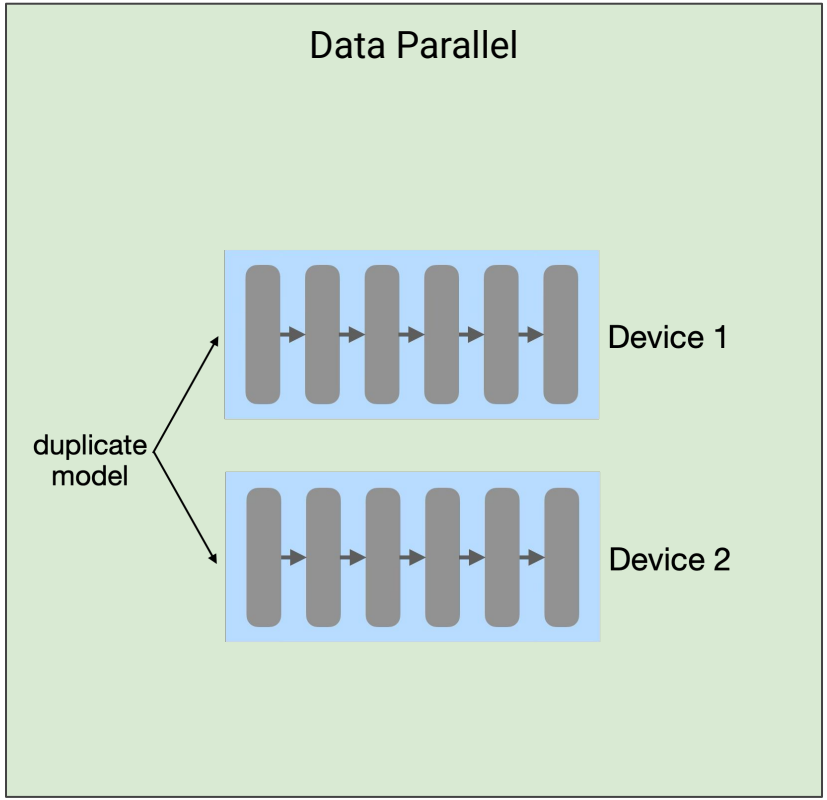
Don't do this (iterate through all of B for each row of C)

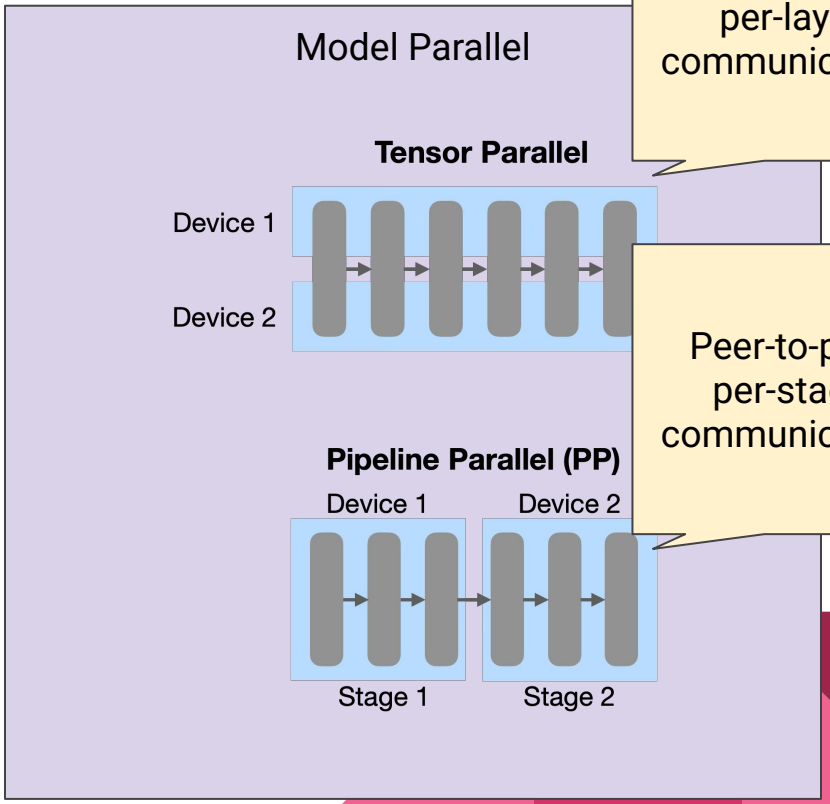
Compute C in "tiles" (small tiles -> more reuse -> more cache friendly)

Now what?

We got our MatMuls running on GPUs. How do we run GPT in parallel?

The 3 types of parallelism in ML

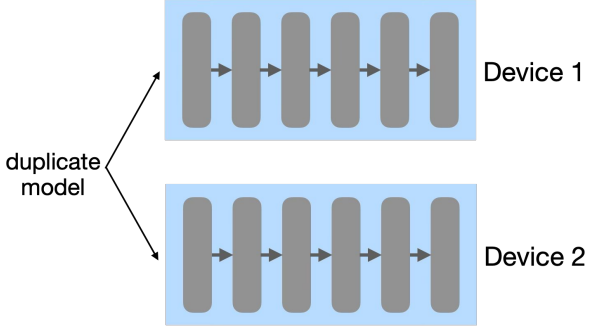




Collective per-layer communication

Peer-to-peer per-stage communication

Data Parallel

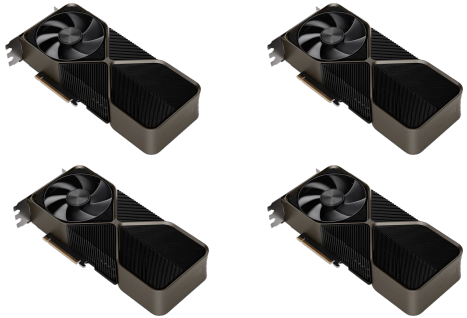


"Scaling out"

If we have a bunch of inputs that we want the LLM to process…

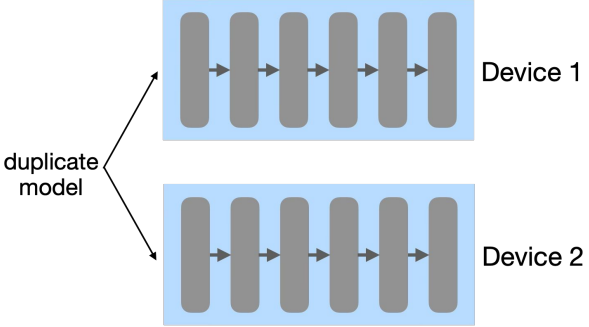
"What is the solution to hw8?"
"Help me write an email asking for an extension."
"Can you forge a doctor's note?"

Run in parallel



Each GPU maintains a full copy of the model
Lots of redundancy… but it scales!

Data Parallelism



Data Parallelism

- Increases throughput, equal latency
- Weights are duplicated, scales embarrassingly (linear)

Model Parallel

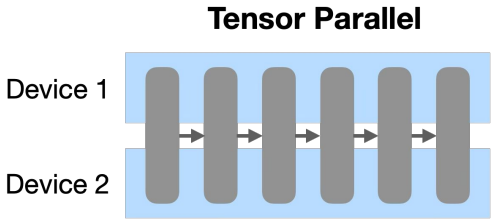
What about parallelizing within one model? Can we avoid the redundant weight matrix duplications?

Model Parallel - Shard the weight/parameter matrices across multiple GPUs

Two approaches:

- Tensor Parallelism
- Pipeline Parallelism

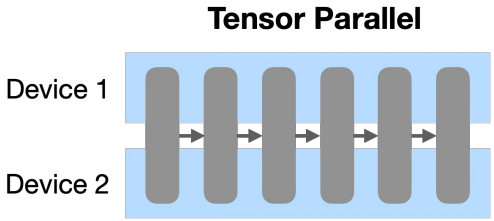
Tensor Parallelism

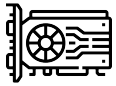


Take each matrix, put half of it on GPU 1, and the other half on GPU 2

Is it that easy?

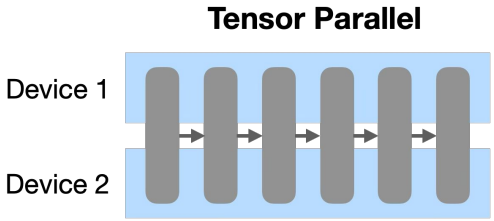
Tensor Parallelism





$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} W_{Q1} & W_{Q2} \\ W_{Q3} & W_{Q4} \end{bmatrix} = \begin{bmatrix} q_1 W_{Q1} + q_2 W_{Q3} & q_1 W_{Q2} + q_2 W_{Q4} \end{bmatrix}$$

Tensor Parallelism



GPU-1

$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} W_{Q1} & \\ W_{Q3} & \end{bmatrix} = \begin{bmatrix} q_1 W_{Q1} + q_2 W_{Q3} & \end{bmatrix}$$

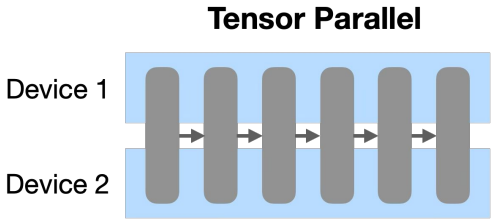
This MatMul output goes into the next MatMul input!

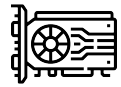
GPU-2

$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} & W_{Q2} \\ & W_{Q4} \end{bmatrix} = \begin{bmatrix} & q_1 W_{Q2} + q_2 W_{Q4} \end{bmatrix}$$

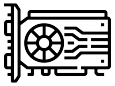
If we have an incomplete input, we can't perform our half of the next MatMul!

Tensor Parallelism - AllGather





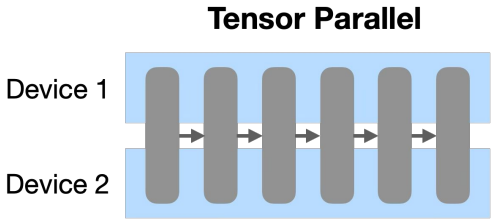
$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} W_{Q1} & \\ W_{Q3} & \end{bmatrix} = \begin{bmatrix} q_1 W_{Q1} + q_2 W_{Q3} & q_1 W_{Q2} + q_2 W_{Q4} \end{bmatrix}$$



$$\begin{bmatrix} q_1 & q_2 \end{bmatrix} \begin{bmatrix} & W_{Q2} \\ & W_{Q4} \end{bmatrix} = \begin{bmatrix} q_1 W_{Q1} + q_2 W_{Q3} & q_1 W_{Q2} + q_2 W_{Q4} \end{bmatrix}$$

Network-bound operation

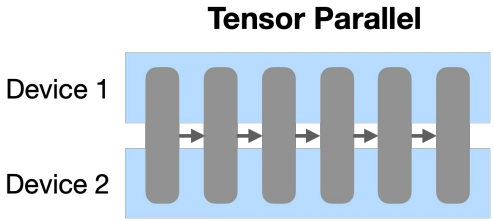
Collective Operations



- AllGather
- Reduce
- Scatter
- All-to-all
- …

Can combine two and optimize, e.g. ReduceScatter

Tensor Parallelism - ReduceScatter

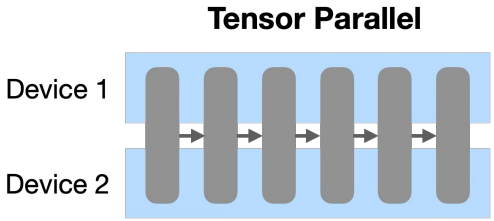


$$\begin{bmatrix} M_1 & M_2 \end{bmatrix} \begin{bmatrix} D_1 & D_2 \\ D_3 & D_4 \end{bmatrix}$$

$$\left[\begin{array}{c:c} M_1D_1 + M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

What if M_1 and M_2 exist on different machines?

Tensor Parallelism - AllReduce



$$\left[\begin{array}{c:c} M_1D_1 + M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

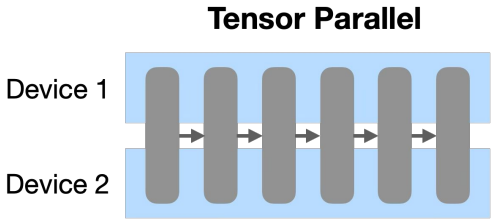
GPU 1

$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{c:c} M_1D_1 & M_1D_2 \end{array}\right]$$

GPU 2

$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{c:c} M_2D_3 & M_2D_4 \end{array}\right]$$

Tensor Parallelism - AllReduce ReduceScatter +



$$\left[\begin{array}{c:c} M_1D_1 + M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

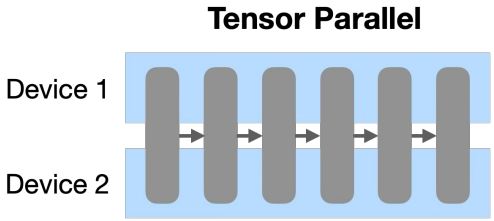
GPU 1

$$M_1 \begin{bmatrix} D_1 & D_2 \end{bmatrix} = \left[\begin{array}{c:c} M_1D_1 & M_1D_2 \end{array}\right]$$

GPU 2

$$M_2 \begin{bmatrix} D_3 & D_4 \end{bmatrix} = \left[\begin{array}{c:c} M_2D_3 & M_2D_4 \end{array}\right]$$

Tensor Parallelism - AllReduce
ReduceScatter + AllGather



$$\left[\begin{array}{cc} M_1D_1 + M_2D_3 & M_1D_2 + M_2D_4 \end{array}\right]$$

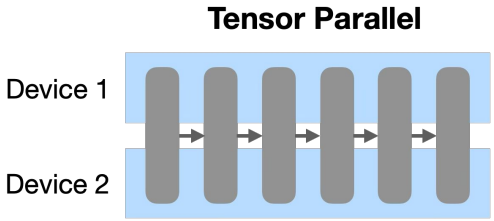
GPU 1

$$M_1 \left[\begin{array}{cc} D_1 & D_2 \end{array}\right] = \left[\begin{array}{cc} M_1D_1 + M_2D_3 & \\ & \end{array}\right]$$

GPU 2

$$M_2 \left[\begin{array}{cc} D_3 & D_4 \end{array}\right] = \left[\begin{array}{cc} & M_1D_2 + M_2D_4 \end{array}\right]$$

Tensor Parallelism - AllReduce
ReduceScatter + AllGather



$$\left[M_1D_1 + M_2D_3 \;\vdots\; M_1D_2 + M_2D_4\right]$$

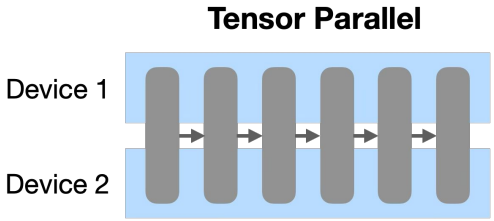
GPU 1

$$M_1 \left[D_1 \quad D_2\right] = \left[M_1D_1 + M_2D_3 \;\vdots\; M_1D_2 + M_2D_4\right]$$

GPU 2

$$M_2 \left[D_3 \quad D_4\right] = \left[M_1D_1 + M_2D_3 \;\vdots\; M_1D_2 + M_2D_4\right]$$

One layer now consists of:



6 matrix multiplies

2 AllGathers

1 AllReduce (roughly 2 more AllGathers)

Will this even be faster than just running it on one machine?

It depends!
How fast is your compute?
How fast is your network?

Data vs Tensor Parallelism

Data Parallelism

- Increases throughput, equal latency
- Weights are duplicated, scales embarrassingly (linear)

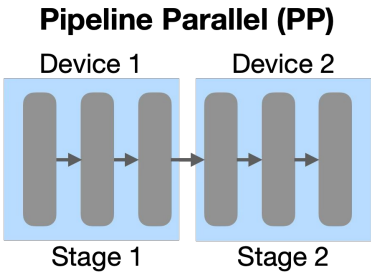
Tensor Parallelism

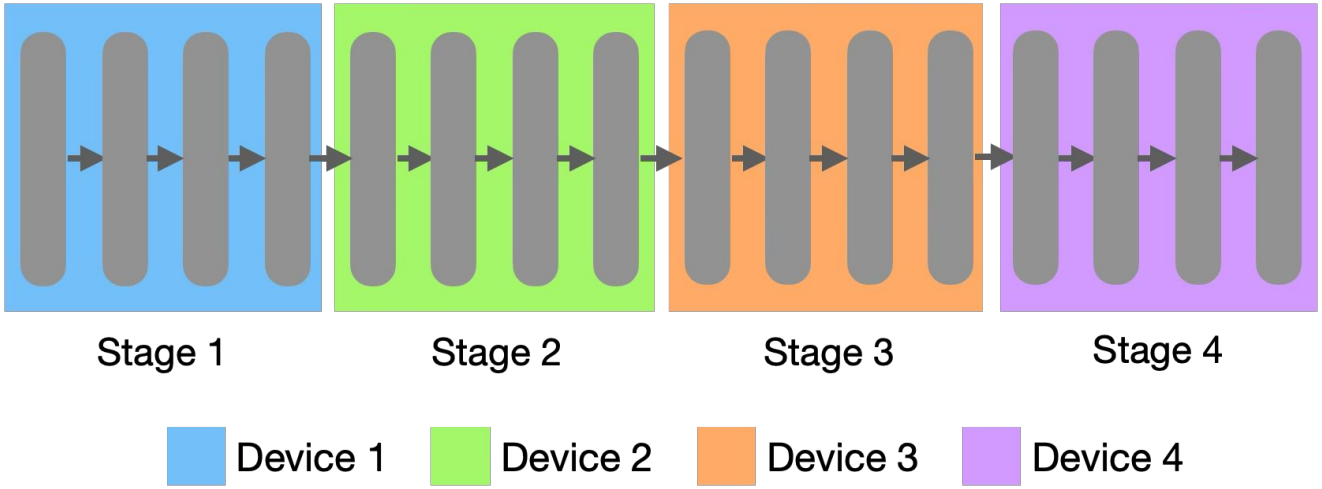
- (Increases throughput, lower latency) iff network is sufficiently fast
- Weights are distributed, scales exponentially

When is the network sufficiently fast?

- GPUs on the same motherboard (NVLink)
- Machines on the same server rack
- Probably not any other time

Pipeline Parallelism





Data vs Tensor vs Pipeline Parallelism

Data Parallelism

- Increases throughput, equal latency
- Weights are duplicated, scales embarrassingly (linear)

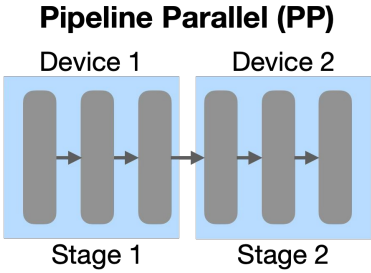
Tensor Parallelism

- (Increases throughput, lower latency) iff network is sufficiently fast
- Weights are distributed, scales exponentially

Pipeline Parallelism

- Increases throughput, higher latency
- Weights are distributed, scales embarrassingly (linear)

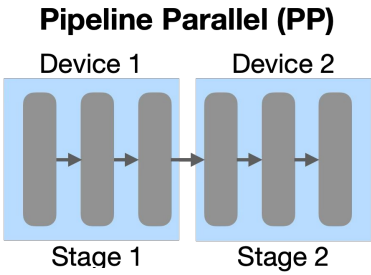
Pipeline Parallelism - Training



PP is simple when we are doing “inference”: going forward through the model

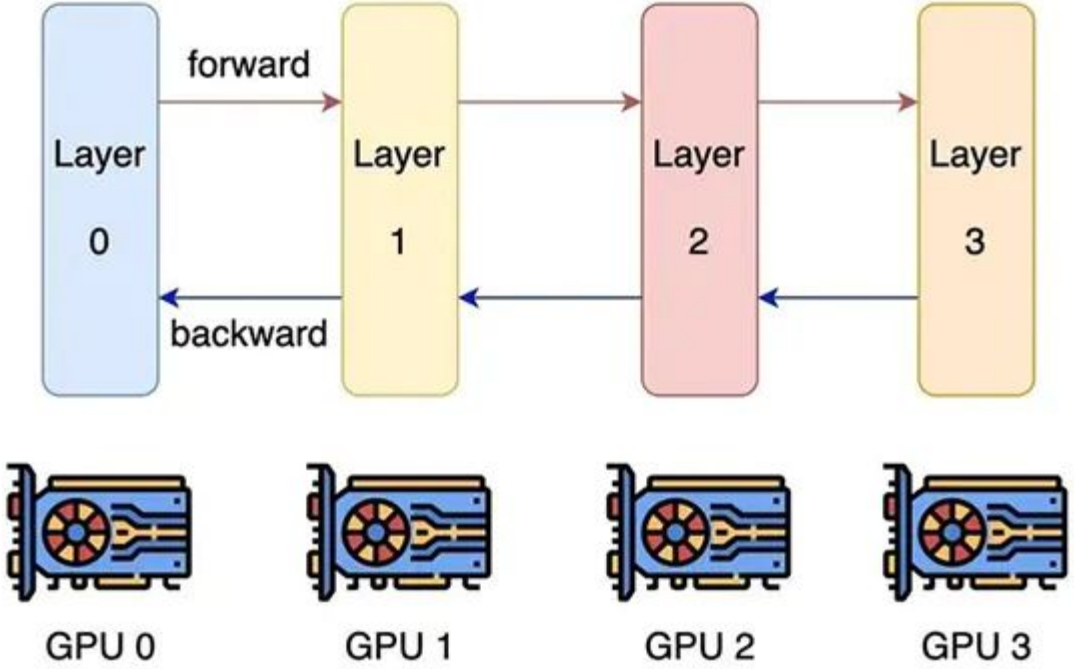
PP becomes a lot more tricky when doing training: going forward and then backwards through the model

Pipeline Parallelism - Training

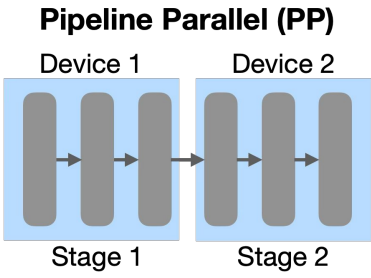


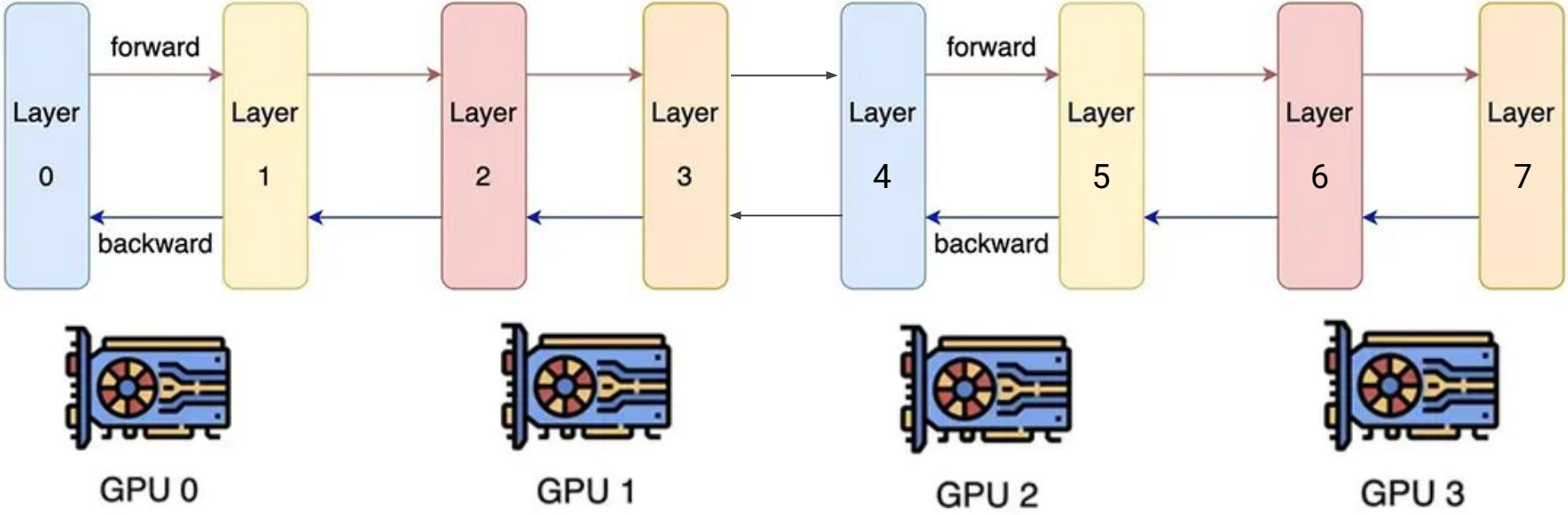
Training involves going backwards through the model to update weights!

Trivial in other parallelism, not so much in pipelining…

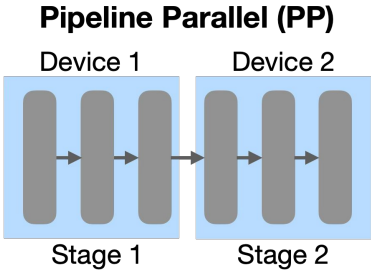


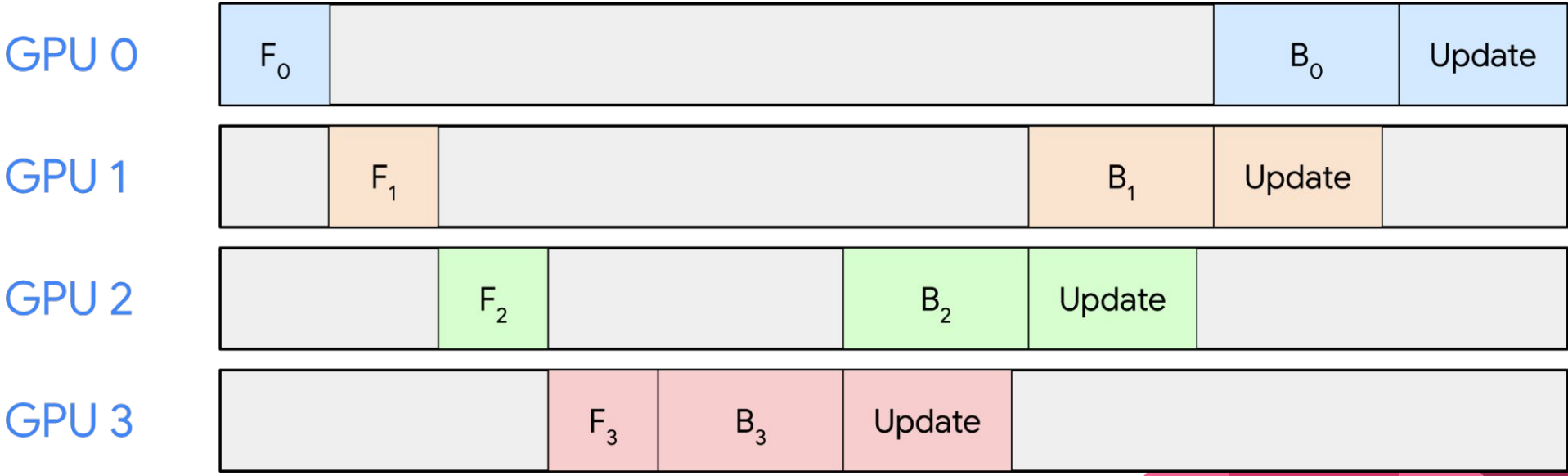
Pipeline Parallelism - Training



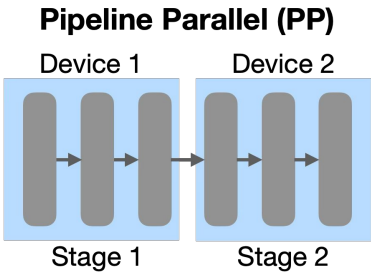


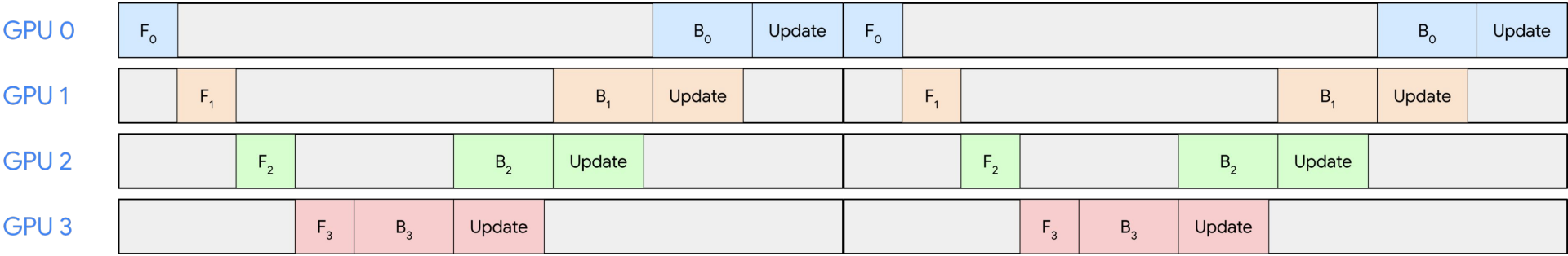
Pipeline Parallelism - Training



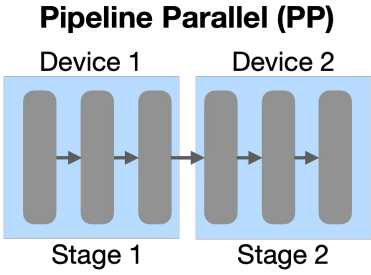


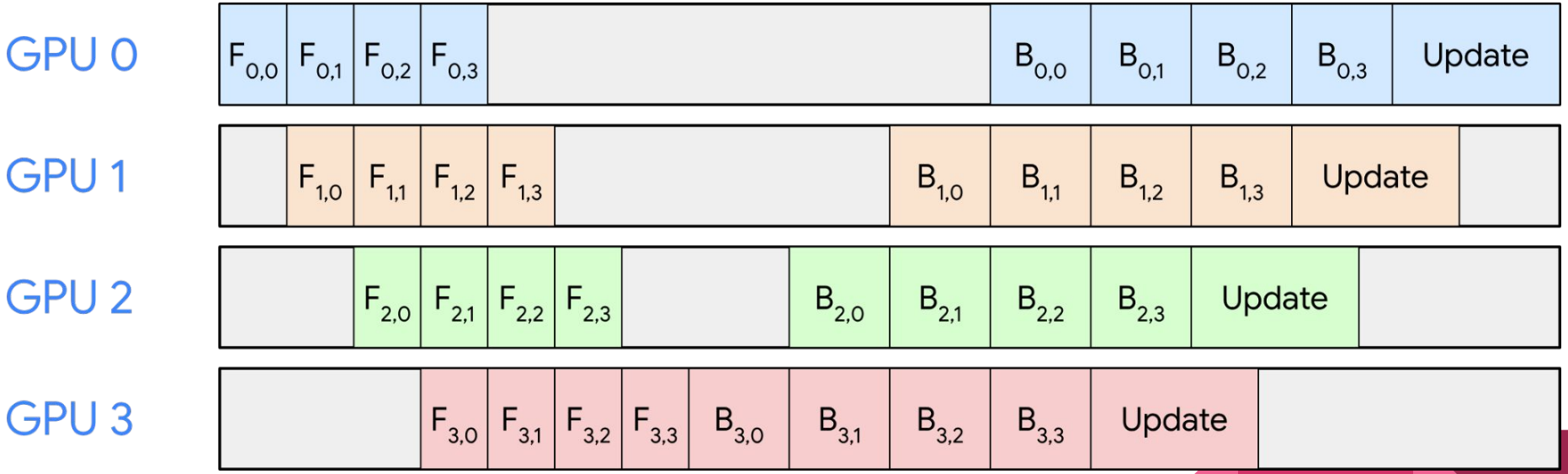
Pipeline Parallelism - Training



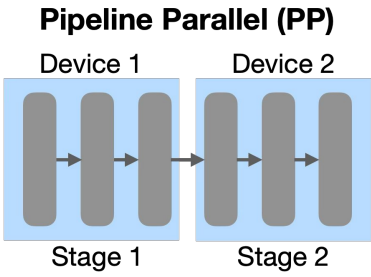


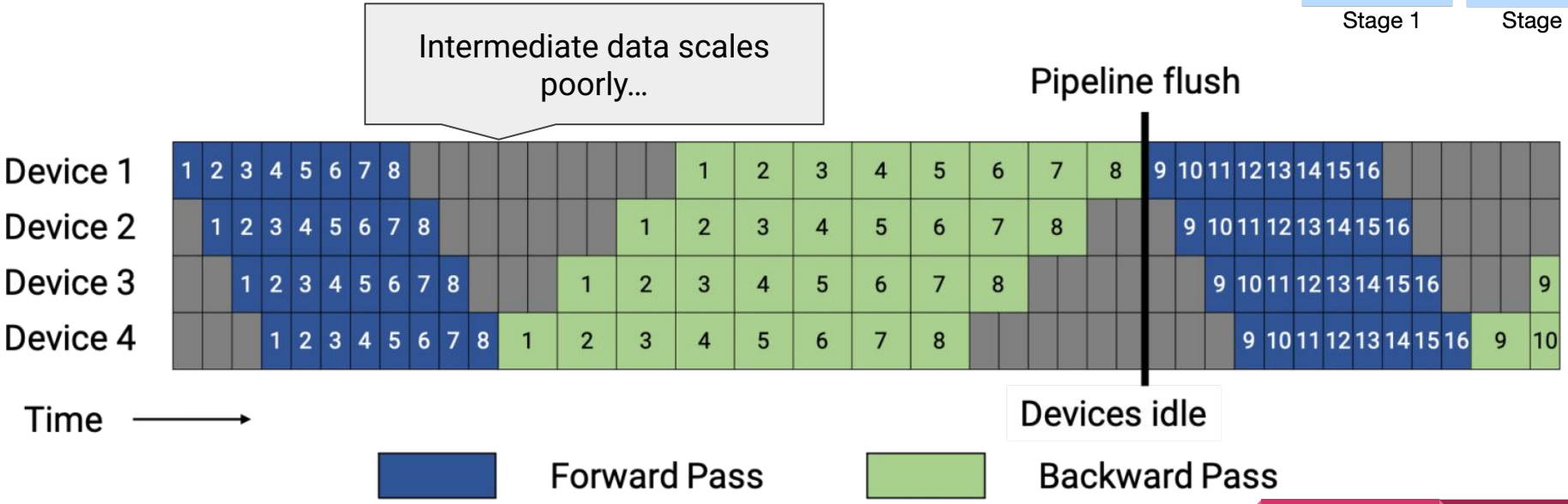
Pipeline Parallelism - Training



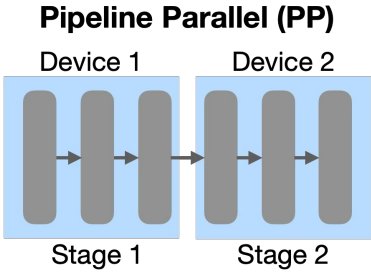


Pipeline Parallelism - Training





Pipeline Parallelism - Training



Device 1 idle for a while…

Pipeline flush

Device 1
Device 2
Device 3
Device 4

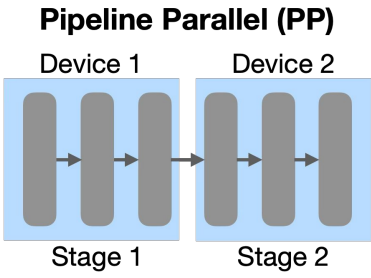
Devices idle

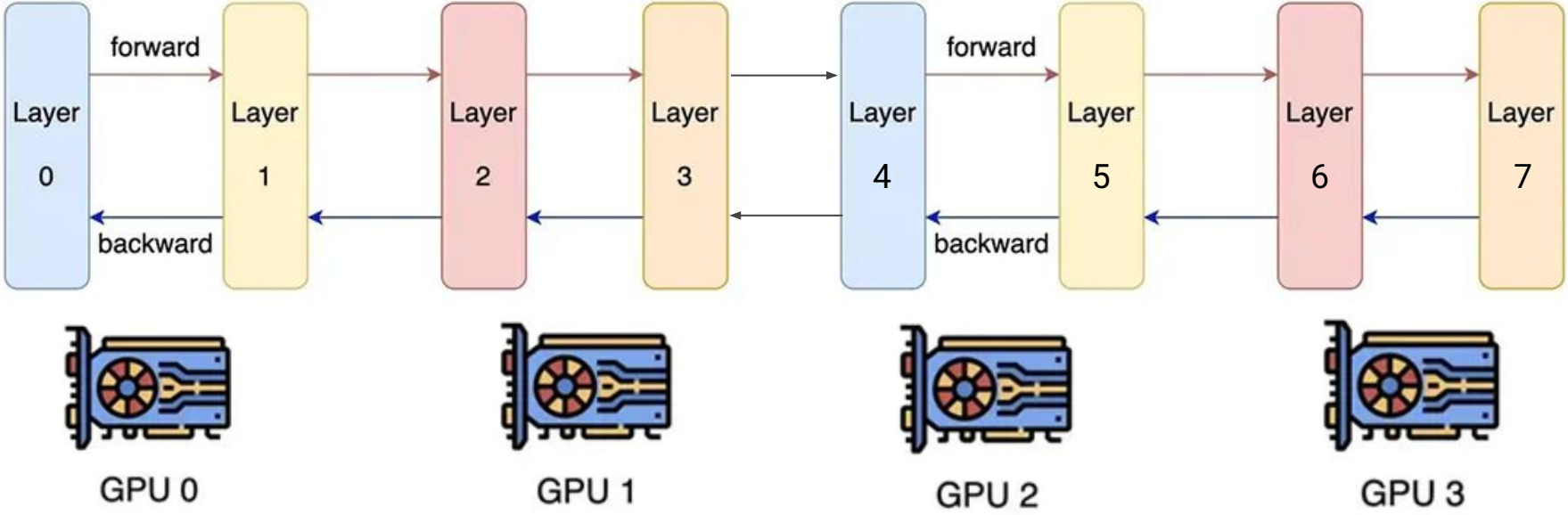
Time

Forward Pass

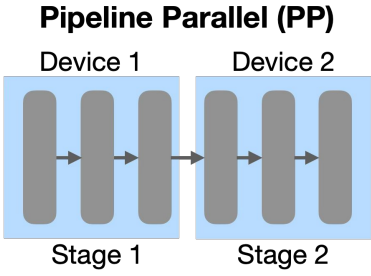
Backward Pass

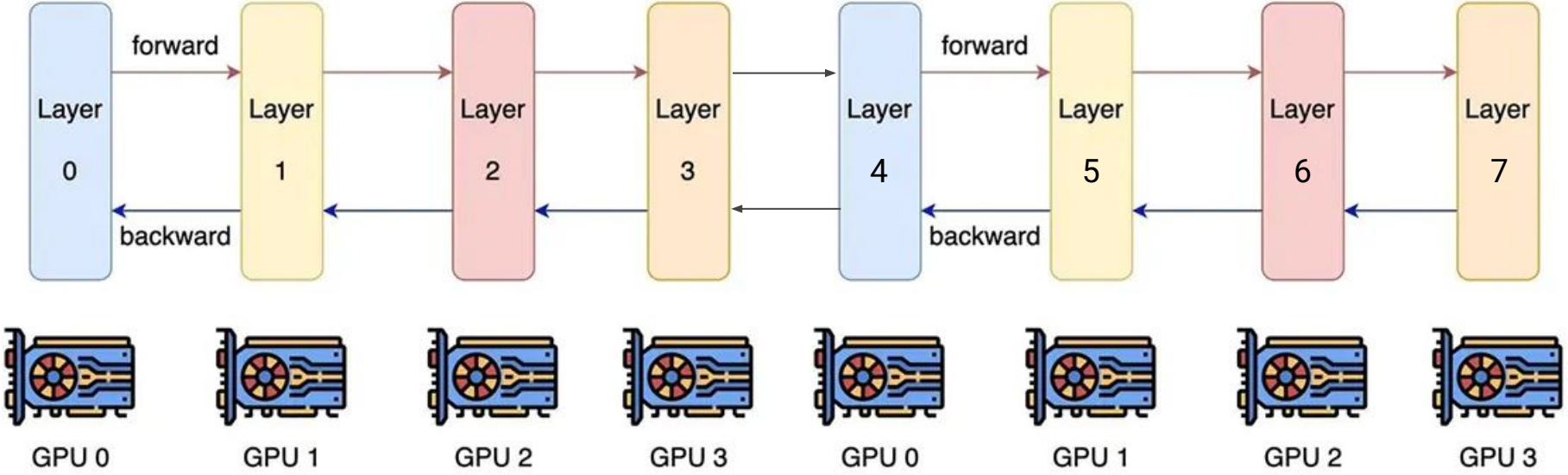
Pipeline Parallelism - Training



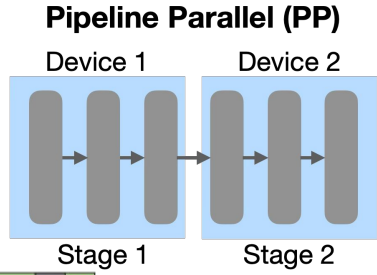


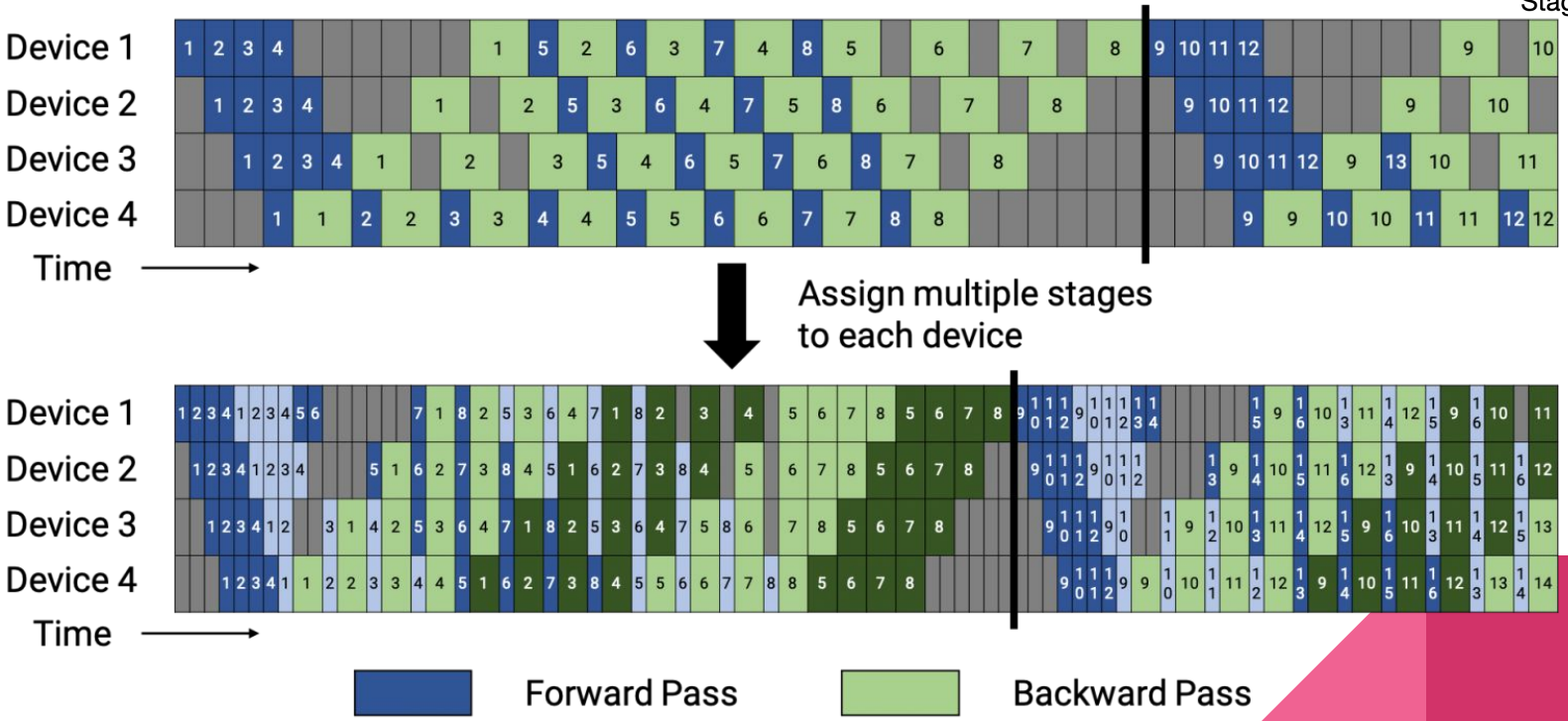
Pipeline Parallelism - Training





Pipeline Parallelism - Training





All of the parallelisms are used at the same time

- Data parallel to scale out as much as possible
- Model parallel to make it possible to have trillions of parameters
  - Tensor parallelism within a super fast interconnect
  - Pipeline parallelism for the remaining sharding necessary

Quiz on Gradescope (11:14-11:20)

Quiz on Gradescope (11:14-11:20)

1. Which statement is **false** about thread pools and async/await?

2. Say you are implementing MapReduce on a cluster of GPUs instead of CPUs. Which part of the computation fits most cleanly onto a SIMD/SIMT model?